Report

Bluefin CLMM — Comprehensive Formal Verification Report

by Asymptotic

Overview

This report documents formal verification of the Bluefin (𝕏) CLMM on Sui. Bluefin CLMM is a concentrated liquidity market maker built on the Uniswap V3 model, where liquidity providers allocate capital within price ranges bounded by discrete tick boundaries. Pricing is tick-based over a fixed-point arithmetic core, swaps settle through a receipt-pattern balance model, and the protocol embeds its own time-weighted price oracle.

We specify and verify the core mechanisms governing pool, position, and swap interactions, together with the concentrated-liquidity math layer that underpins swap solvency and liquidity conversion. Coverage extends to tick state, the embedded TWAP oracle, the reward-emission accounting, and the capability-gated administrative surface.

TABLE OF CONTENTS

How We Work

Report Scope

Verification Highlights

Methodology

Reproducibility

Operational and Platform Assumptions

Verification Details

- Pool Lifecycle
- Flash Swap and Repayment
- Position Accounting
- Reward Emission and Accrual
- Tick State Primitives
- TWAP Oracle
- Administrative Governance
- Gateway Entry Points

Pool Swap Result Accessors
Concentrated Liquidity Math
Tick Math and Signed Integers
Lean 4 proofs (Sui Prover Lean backend)
Proof architecture
Assumptions
Arithmetic Operations

Conclusion and Continual Assurance

How We Work

1. Understand the protocol and identify what could go wrong.
2. Prove the invariants that matter: the ones that prevent failures, not the easy ones. We go beyond shallow verification (model-checking, easy SMT solving) and use hand-crafted Boogie/smtlib and Lean proofs for the hardest properties.
3. Publish clear reports so users can understand what is safe about a protocol.

Report Scope

We cover the Bluefin CLMM Core contracts. Coverage spans pool state and operations, position and tick accounting, the reward-emission accounting, the TWAP observation buffer, the global configuration and capability-gated administrative surface, and the public gateway used by integrators. It also reaches the foundational math: the concentrated-liquidity layer, the tick-to-price map, and the signed-integer arithmetic helpers. The exact package and source commit are identified in Reproducibility.

We prove that pool creation initializes a new pool deterministically and that subsequent state transitions preserve the structural invariants. The flash-swap path is shown to bound the post-swap price between its starting price and the caller's limit, to keep fee growth and protocol fees monotonically non-decreasing on the active side, and to issue a receipt that records the correct payment amount and side. Position lifecycle operations update pool and position state coherently.

The tick state primitive maintains gross and net liquidity consistently, and the TWAP observation buffer preserves the cardinality, indexing, and cumulative-value invariants required for sound oracle readings. The concentrated-liquidity math layer is proven to never let a single swap step pay out more value than it takes in, to convert deposits to liquidity and back without overcrediting, and to round consistently. Every administrative operation gates its effect on a valid capability and produces the expected change to the global configuration or pool object.

Out of scope: off-chain infrastructure, the accuracy of external price feeds, and UI and frontend code.

A distinct boundary is *what is claimed* about the composed swap and liquidity-provision entry points, as opposed to what is out of scope. Those entry points are specified and proven, but their proofs are conditioned on the call completing rather than characterising exactly when they revert. See Scope of abort coverage below. The building blocks of those paths are independently verified.

Verification Highlights

Swap Solvency. The single-step swap kernel is proven to never pay out more value than it takes in: valuing the output token at the pool price and comparing against the input, the inequality holds for every swap step over any valid price range. Liquidity conversion is proven to round in the solvent direction and to round-trip without crediting a depositor more than their deposit backs. See Concentrated Liquidity Math.

Pool Lifecycle. Pool creation is proven to initialize every field of the new pool deterministically, from its fee configuration and starting price down to zeroed liquidity, balances, fee growth, and sequence number. Position lifecycle entry points (e.g. `open_position`) are proven to preserve every pool field outside the position index counter, so opening or closing an empty position does not perturb the pool's price, liquidity, fees, or balances. See Pool Lifecycle.

Tick State Consistency. The `tick::update` primitive is proven to correctly initialize, update, and remove tick boundaries while maintaining the signed net liquidity invariant of the Uniswap V3 model. The `tick::cross` primitive is proven to flip the fee, reward, and timing accumulators outside the boundary while leaving gross and net liquidity unchanged, and to return the boundary's net liquidity as the active-liquidity delta. See Tick State Primitives.

Position Accounting. Position creation initializes liquidity to zero and binds the position to its pool, fee rate, and tick bounds. The position update primitive is proven to recompute accrued fees using the wrapping fee-growth delta and the position's prior liquidity, to apply the signed liquidity delta, and to refresh the recorded fee-growth and reward-growth snapshots in lockstep. See Position Accounting.

TWAP Oracle Soundness. The observation manager's `grow` operation is proven to extend the buffer to the requested cardinality without disturbing the active index or the existing observation contents. The `update` operation is proven to advance the index modulo the active cardinality, expand the cardinality only when the next slot is reached, and write a new observation whose cumulative values are computed from the previous observation and the elapsed time. The `observe_single` lookup is proven to return the exact stored observation when the target timestamp matches an existing entry. See TWAP Oracle.

Administrative Governance. Every capability-gated administrative operation is proven to require a synchronized config version, to apply its intended effect, and to leave unrelated configuration unchanged. Capability transfers are proven via ghost-state to dispatch the cap to the intended recipient. See Administrative Governance.

Flash Swap Repayment. The receipt-pattern settlement is verified end to end. `repay_flash_swap` is proven to credit the pool by exactly the receipt amount on the correct side, leave the opposite side untouched, preserve every other pool field, bump the sequence number, and clear the paused flag. The price-bound, receipt-fidelity, and fee-monotonicity properties of the opening `flash_swap` call are proven for executions that complete; the conditions under which it reverts are not characterised. See Flash Swap and Repayment.

Tick Math and Signed Arithmetic. The tick-price conversions are proven correct, monotonic, and invertible via Lean 4 interactive proofs, with exhaustive verification over all 887,273 valid ticks. The signed-integer helper libraries are proven to agree with the reference signed-integer semantics on every operation, with overflow asserts that pin each result to the signed range. See Tick Math and Signed Integers.

Arithmetic Soundness. The `add_delta` helper underpinning every liquidity mutation is proven to assert non-underflow on subtraction and non-overflow on addition, to compute the signed sum exactly, and to bound the result strictly below `u128::MAX`. See Arithmetic Operations.

Methodology

Proof structure. Each specification mirrors the signature of its target function. The spec body establishes preconditions (`asserts` -- verified

by the prover, `requires` -- assumed, typically for overflow bounds that are structurally guaranteed but expensive to re-derive), invokes the target, then states postconditions (`ensures`) that the prover must discharge.

Abort modeling. Where feasible, specifications fully model abort conditions using `asserts` preconditions that the prover verifies, proving both that the function succeeds when preconditions hold and aborts otherwise. Example abort paths covered include version-mismatch checks against the global config, paused-pool guards, sqrt-price and tick-range bounds, and overflow checks in liquidity arithmetic.

Abort handling. For composed user-facing operations where exhaustive abort modeling is impractical, specifications verify properties of successful executions only. Abort conditions are still enforced by implementation-level assertions that revert the transaction. The spec proves correct state updates on success.

Access control. Privileged operations take capability objects (e.g. `AdminCap`, `ProtocolFeeCap`) as parameters. In Move, function signatures are enforced at the bytecode level: a caller cannot invoke a privileged function without possessing the required capability. This is a language-level guarantee, not a property the prover re-derives.

Spec Composition. Larger composed specs factor through per-module spec libraries so that callees are summarised by their proven specs rather than re-inlined, making specs over composed call graphs tractable.

Ghost State. State and effects that the prover cannot reason about directly, such as dynamic table-backed storage and transfer-recipient witnessing, are modeled with ghost variables.

Scope of abort coverage. Most specifications in this package carry an exact abort characterisation: the function reverts if and only if the stated conditions are violated. A minority instead claim correctness on completion — if the call returns, the postconditions hold — without characterising which inputs cause a revert. This applies to the relational lemmas in the math layer, where it is the natural form and the companion functional specifications carry exact abort coverage, and to the composed swap and liquidity-provision entry points. For the latter it is a deliberate, evidenced decision rather than an unfinished proof: an earlier revision asserted exact abort equivalence over those paths, and that

assertion was proven false by machine-checked counterexample. Claiming it would be a false statement. Their building blocks are independently verified, and all such cases are tracked in the verification dashboard.

Reproducibility

Verified package. The subject of this report is the Bluefin CLMM package as it is deployed on Sui mainnet today.

Package	`BluefinSpot` — Move Registry id `@bluefinprotocol/bluefin-sp`
Package id (version 17)	`0xd075338d105482f1527cbfd363d6413558f184dec36d9138a70261e8`
Original published id	`0x3492c874c1e3b3e2984e8c41b589e642d4d0a5d6459e5a9cfc2d52fd`
On-chain version	17

Source. The protocol sources this verification targets are the Bluefin Spot repository at commit `925896ef38be892975a82e8211352b6d2cf08b66` (`fireflyprotocol/bluefin-spot-contracts`).

- Sui Prover version: `1.5.3`
- `sui` binary used for source verification: `1.62.0-74e8414a4986`

Operational and Platform Assumptions

These proofs rely on the correctness of the Sui execution environment: the Move VM, bytecode verifier, and runtime resource/capability enforcement are assumed to behave according to their specifications. Standard network conditions for protocol operation are assumed, including validator correctness, epoch progression, and chain liveness.

The concentrated-liquidity math, the tick-to-price map, and the signed-integer helpers are verified directly rather than assumed, so the per-step swap solvency, the liquidity-conversion rounding direction, and the price-range bounds rest on proofs. The proofs do assume the underlying `u256` fixed-point arithmetic behaves as modeled, and a small number of the math specs leave the rounding division uninterpreted, proving the rounding direction rather than the exact quotient.

`AdminCap` and `ProtocolFeeCap` are assumed held and operated by the intended governance actors, and the pool-creation fee mechanism is assumed to be configured per the implementation's own assertions.

The TWAP oracle's soundness is proven against the structural invariants of the observation buffer (cardinality bounds, index wrap-around, cumulative monotonicity within the i64/u256 ranges). The semantic interpretation of the resulting time-weighted average price as a fair market reading is a property of how integrators consume the oracle, not of the on-chain state itself.

The guarantees here cover modeled contract behavior, including explicitly modeled abort paths and proven postconditions. They do not cover off-chain infrastructure, operational processes, or source-level properties not expressed in the specifications.

Verification Details

Pool Lifecycle

Pool creation is the single point at which a new pool's state is established, and every subsequent operation assumes the post-creation invariants. The `create_pool_internal` spec proves that after creation:

$$\texttt{pool.fee_rate} = \texttt{fee_rate}$$
$$\texttt{pool.current_sqrt_price} = \texttt{current_sqrt_price}$$
$$\texttt{pool.current_tick_index} = \texttt{tick_math::get_tick_at_sqrt_p}$$
$$\texttt{pool.liquidity} = 0 \qquad \texttt{pool.position_index} = 0$$
$$\texttt{pool.sequence_number} = 0$$
$$\texttt{pool.fee_growth_global_a} = \texttt{pool.fee_growth_global_b} = 0$$
$$\texttt{pool.protocol_fee_a} = \texttt{pool.protocol_fee_b} = 0$$
$$\texttt{balance}(\texttt{pool.coin_a}) = \texttt{balance}(\texttt{pool.coin_b}) = 0$$
$$\texttt{pool.is_paused} = \texttt{false}$$

The spec asserts the input ranges that must hold for a sound pool: tick spacing within the configured maximum, fee rate within the configured maximum, starting sqrt price within $[\texttt{min_sqrt_price}, \texttt{max_sqrt_price})$, and the two coin types distinct.

`open_position` and `close_position_v2` are proven to preserve the rest of the pool. Opening a position bumps the position index by one and produces a `Position` whose pool id, tick bounds, fee rate, and zero starting liquidity all derive from the pool's current state. The pool's price, tick index, liquidity, balances, fee growth, and protocol-fee accumulators are unchanged. Closing a position with zero liquidity and zero outstanding fees is proven to be a no-op on the pool's balances, price, and fee state.

Specifications: Pool Lifecycle

`pool::create_pool_internal`
`pool::open_position`
`pool::close_position_v2`
`pool::collect_fee`
`pool::increase_observation_cardinality_next`
`pool::set_manager`
`pool::set_protocol_fee_amount`
`pool::increase_reward_coin_reserves`
`pool::increase_sequence_number`

Flash Swap and Repayment

Bluefin's swap entry point is the receipt-pattern `flash_swap`. The pool releases the requested side as a `Balance` and a `FlashSwapReceipt`, atomically pausing the pool, and the caller must invoke

`repay_flash_swap` in the same transaction to settle the debt and unpause.

The settlement half of this protocol, `repay_flash_swap`, is proven with exact abort coverage. The opening `flash_swap` call is proven for executions that complete; the conditions under which it reverts are not characterised, for the reasons given under Scope of abort coverage. Its building blocks (the swap-step solvency kernel, the tick primitives, and the price map) are independently verified. The properties stated here are proven of every completing execution.

Price bounds. The specification bounds the post-swap price within the protocol's global range and between the caller's limit and the starting price:

$$\texttt{min_sqrt_price} \le \texttt{pool.current_sqrt_price} \le \texttt{max_sqrt_pri}$$

$$\texttt{sqrt_price_max_limit} \le \texttt{pool.current_sqrt_price}_{\texttt{after}} \le \quad (\text{if } a2b)$$

$$\texttt{current_sqrt_price}_{\texttt{before}} \le \texttt{pool.current_sqrt_price}_{\texttt{after}} \le \quad (\text{if } b2a)$$

The directional bound is the property a price-oracle integrator relies on: a swap cannot push the pool past the caller's limit, and cannot retrograde against the trade direction.

Receipt fidelity. The specification binds the post-call obligation to the correct pool, direction, and amount:

$$\begin{aligned} \texttt{receipt.pool_id} &= \texttt{object::id}(\texttt{pool}) \\ \texttt{receipt.a2b} &= a2b \\ \texttt{receipt.pay_amount} &\le \texttt{amount} \quad (\text{by amount in}) \end{aligned}$$

When the caller specifies an output amount (`by_amount_in = false`), the receipt's pay amount is bounded above by the available reserve on the side being paid.

Active-side fee monotonicity. Fees can only accumulate on the side being swapped:

$$\texttt{pool.fee_growth_global_coin_a}_{\texttt{after}} \geq \texttt{fee_growth_a}_{\texttt{before}}$$
$$\texttt{pool.protocol_fee_coin_a}_{\texttt{after}} \geq \texttt{protocol_fee_a}_{\texttt{before}} \quad (\text{if } a2b)$$

The inactive-side fee growth and protocol fee accumulator are specified to equal their pre-call values, so a swap in one direction cannot perturb the other side's fee bookkeeping.

Balance accounting. The released balance and the pool's remaining balance partition the pre-call inactive-side reserve:

$$\texttt{balance}(\texttt{released}) + \texttt{balance}(\texttt{pool.coin_x}_{\texttt{after}}) = \texttt{coin_x}_{\texttt{before}}$$

The active side's pool balance is unchanged at this stage. The caller is expected to deposit the receipt amount on the active side via `repay_flash_swap`, which is fully proven: it credits the pool by exactly the receipt amount on the correct side, leaves the other side's balance untouched, preserves every other pool field (price, tick, liquidity, fee growth, protocol fees, reward configuration), bumps the sequence number, and clears the paused flag. `swap_pay_amount` is proven to read the receipt's recorded payment exactly.

Specifications: Flash Swap and Repayment

`pool::repay_flash_swap`

`pool::swap_pay_amount`

Position Accounting

A `Position` is a non-fungible NFT recording one liquidity provider's stake in a single tick range. The `position::update` primitive is the funnel through which every liquidity change, fee accrual, and reward refresh passes.

Fee accrual using wrapping deltas. Fees outside the tick range are tracked as monotonically increasing globals at the pool level. When a position is refreshed, the per-unit-liquidity fee growth inside the range is differenced against the position's last snapshot using `math_u128::wrapping_sub`, multiplied by the position's prior liquidity, and scaled by Q_{64}:

$$\Delta f_x = \texttt{wrapping_sub}(\texttt{fee_growth_inside_x}, \texttt{fee_growth_in}$$

$$\texttt{owed_x} = \texttt{mul_div_floor}(\Delta f_x,\ \texttt{liquidity}_{\texttt{before}},\ Q_{64})$$

The spec asserts that the scaled product fits in `u128`, that the cast to `u64` is safe, and that the resulting owed fee adds to the prior `token_x_fee` without overflow. The post-call `token_x_fee` is then exactly the prior value plus the owed amount.

Signed liquidity delta. The position's liquidity is updated by a signed `I128` delta with overflow and underflow asserts. When the delta is zero, the spec requires the position to have non-zero prior liquidity, since a fee-refresh of an empty position is forbidden. When the delta is negative, prior liquidity must cover the magnitude. When the delta is positive, the sum must fit in `u128`. The position's fee-growth snapshots are refreshed to the new global insides, and the reward-growth vector is extended to match the position's reward slots.

Construction. `position::new` is proven to bind the position to its pool by id, set the requested tick bounds, copy the pool's fee rate, set liquidity to zero, and produce a fresh position index. Closing an empty position via `position::del` is proven to be a no-op on the surrounding accounting.

Specifications: Position Accounting

`position::new`

`position::update`

`position::del`

`position::is_empty`

`position::liquidity`

`position::lower_tick`

`position::upper_tick`

`position::pool_id`

`position::get_accrued_fee`

`position::set_fee_amounts`

`position::add_reward_info`

`position::coins_owed_reward`

`position::decrease_reward_amount`

`position::get_mutable_reward_info`

`position::reward_infos_length`

`position::update_reward_infos`

`position::create_position_description`

`position::create_position_name`

Reward Emission and Accrual

Pools distribute incentive rewards to liquidity providers in addition to swap fees. Reward state lives in a per-pool vector of `PoolRewardInfo` entries (one per reward coin type) mirrored by a per-position reward vector. The specs prove that emission updates and per-position accrual track these accumulators coherently.

Pool-side accumulation. `update_reward_infos` is proven to advance each reward's per-unit growth accumulator by the emission accrued over the elapsed time, scaled by the pool's active liquidity, and to record the update timestamp. `add_reward_info` is proven to append a new reward type at the end of the vector while preserving every existing entry by index, and `increase_reward_coin_reserves` is proven to credit the reward reserve by exactly the deposited balance. The lookup helpers `find_reward_info_index` and `is_reward_present` are proven to locate a reward type correctly.

Position-side accrual. `position::update_reward_infos` is proven to bring each position reward slot up to the pool's current growth, computing the owed amount from the wrapping growth delta and the position's liquidity. `decrease_reward_amount` is proven to debit the recorded owed amount by exactly the claimed quantity, and `coins_owed_reward` is proven to read it exactly. `pool::get_accrued_rewards` composes these to compute a position's claimable rewards while preserving the pool's price, fees, and balances.

Reward-manager governance. The reward-manager set on the global config gates which accounts may configure pool rewards.

Specifications: Reward Emission and Accrual

`pool::update_reward_infos`

`pool::add_reward_info`

`pool::increase_reward_coin_reserves`

`pool::find_reward_info_index`

`pool::is_reward_present`

`pool::reward_infos_length`

`pool::get_accrued_rewards`

`pool::default_reward_info`

`position::update_reward_infos`

`position::decrease_reward_amount`
`position::coins_owed_reward`
`position::get_mutable_reward_info`
`position::reward_infos_length`
`admin::add_reward_manager`
`admin::remove_reward_manager`
`admin::add_reward_reserves_to_pool`

Tick State Primitives

The tick manager stores per-boundary state in a bitmap-indexed table. The central primitive proven against this storage layout is `tick::update`, which inserts, modifies, or removes a tick boundary in response to a liquidity change. The companion `tick::cross` primitive, which flips the boundary's outside accumulators when a swap step crosses it, is proven with exact abort coverage on the Lean backend.

Update. For an existing tick, gross liquidity moves by the absolute delta and net liquidity moves by the signed delta with sign depending on the boundary kind:

$$\mathtt{tick.liquidity_gross}_{\mathtt{after}} = \mathtt{tick.liquidity_gross}_{\mathtt{before}} \pm |$$

The boolean return value is proven to be exactly the flip indicator: the tick's existence transitioned (`true`) if and only if the tick was either newly created from zero gross liquidity or fully removed when gross liquidity returned to zero. For a fresh tick, the spec asserts the index lies within the global tick range and initializes the tick's stored fields.

Cross. Crossing a tick is specified to leave its index, sqrt price, gross liquidity, and net liquidity unchanged, while flipping the outside accumulators using wrapping subtraction against the current globals:

$$\mathtt{tick.fee_growth_outside_x}_{\mathtt{after}} = \mathtt{wrapping_sub}(\mathtt{fee_growth_}$$
$$\mathtt{return} = \mathtt{tick.liquidity_net}_{\mathtt{before}}$$

Reward-growth outsides, seconds-per-liquidity outside, and the tick-cumulative outside are flipped in the same way against their respective globals, with asserts that the corresponding global value is at least the prior outside value (so the wrap encodes a non-negative delta). The

returned net liquidity is the swap loop's signed adjustment to active liquidity as the price crosses the boundary.

Initialization and accessors. `initialize_manager` is proven to build a tick manager with the requested tick spacing and empty storage. Tick getters, setters, and the boundary-fee/reward derivations are individually proven and serve as building blocks for the composed liquidity-provision specs.

Specifications: Tick State Primitives

`tick::update`

`tick::create_tick`

`tick::remove`

`tick::is_tick_initialized`

`tick::initialize_manager`

`tick::compute_reward_growths`

`tick::get_fee_and_reward_growths_inside`

`tick::get_fee_and_reward_growths_outside`

`tick::fetch_provided_ticks`

`tick::get_tick_from_manager`

`tick::get_mutable_tick_from_manager`

`tick::get_tick_from_table`

`tick::get_mutable_tick_from_table`

`tick::liquidity_gross`

`tick::liquidity_net`

`tick::sqrt_price`

`tick::tick_spacing`

`tick_bitmap::flip_tick`

`tick_bitmap::position`

`tick_bitmap::cast_to_u8`

`tick_bitmap::get_immutable_tick_word`

`tick_bitmap::get_mutable_tick_word`

`tick_bitmap::mod_compress_tick_range`

`bit_math::most_significant_bit`

`bit_math::least_significant_bit`

TWAP Oracle

The observation manager stores a circular buffer of `(timestamp, tick_cumulative, seconds_per_liquidity_cumulative)` observations.

Three primitives carry the soundness of the TWAP reading: `grow` extends the buffer, `update` writes a new observation when at least one second has elapsed, and `observe_single` interpolates a historical value at a requested timestamp.

Cardinality and indexing. `grow` is proven to set the next cardinality to the requested value, leave the active cardinality and index untouched, and append default observations to the storage vector. The spec quantifies over the storage vector to prove that entries outside the newly-grown region are unchanged:

$$\forall i.\, i < \mathbf{old_len} \;\vee\; i \geq \mathbf{new_len} \;\Rightarrow\; \mathbf{obs}[i] \text{ unchanged}$$

`update` is proven to write at index $(\mathbf{obs_index} + 1) \bmod \mathbf{cardinality}_{\mathbf{updated}}$, where `cardinality_updated` expands to `cardinality_next` exactly when the previous index reached the end of the active cardinality. The new observation's timestamp equals the call target, and the cumulative-tick and cumulative-seconds-per-liquidity fields are computed from the previous observation plus the time delta scaled by the current tick and liquidity. The spec carries explicit i64 overflow guards for the tick-cumulative product.

Observation lookup. `observe_single` is proven to return the latest observation's stored values when the target timestamp equals the latest observation's timestamp, and to interpolate between two surrounding observations otherwise. The spec proves the lookup is well-defined when the latest observation is initialized and the target lies in the buffered window, and asserts the corresponding bound when `seconds_ago = 0`.

Specifications: TWAP Oracle

`oracle::initialize_manager`
`oracle::grow`
`oracle::update`
`oracle::observe_single`
`oracle::get_observation`
`oracle::get_surrounding_observations`
`oracle::default_observation`
`oracle::transform`

Administrative Governance

Every administrative entry point sits behind a capability parameter (`AdminCap` or `ProtocolFeeCap`). At the bytecode level, a caller cannot invoke any of these functions without holding the capability object, so capability possession is a language-level guarantee rather than a property the prover re-derives. On top of that, each spec asserts the global config's version equals the package version, so callers cannot apply governance through a stale config.

Configuration updates. The reward-manager mutators and the lower-level config setter are specified to insert, remove, and update the reward-manager set without disturbing the tick range or config version.

Pool governance. Pool-governance entry points are specified to apply their intended effect and otherwise preserve pool state.

Fee and capability transfer. Fee-collection and capability-transfer entry points are specified to debit the appropriate accumulators and dispatch assets to the destination address. Version-update entry points are specified to bump the config version atomically.

Specifications: Administrative Governance

`admin::add_reward_manager`

`admin::remove_reward_manager`

`admin::add_reward_reserves_to_pool`

`admin::claim_pool_creation_fee`

`admin::claim_protocol_fee`

`admin::increase_observation_cardinality_next`

`admin::set_pool_manager`

`admin::transer_admin_cap`

`admin::transer_protocol_fee_cap`

`admin::update_pool_pause_status`

`admin::update_protocol_fee_share`

`admin::update_supported_version`

`config::set_reward_manager`

`config::get_tick_range`

`config::verify_version`

Gateway Entry Points

The `gateway` module exposes the public surface used by integrators. Its specs compose the pool-level specs above, threading transfer ghosts so

that the recipient of each output coin can be witnessed.

Gateway entry points compose the pool-level specs above. The fee-collection, reward-collection, position-close, and pool-creation entry points are proven with exact abort coverage, with postconditions binding dispatched coin balances to the requested recipient. The composed swap and liquidity-provision entry points — including the routing entry point — are proven for executions that complete; their revert conditions are not characterised, for the reasons given under Scope of abort coverage. The pool-level effects they compose are independently verified. The full set is tracked in the verification dashboard.

Specifications: Gateway Entry Points

`gateway::collect_fee`

`gateway::route_swap`

Pool Swap Result Accessors

Each step of the swap loop produces a `SwapResult` accumulator that records the running `amount_specified_remaining`, `amount_calculated`, `fee_amount`, `protocol_fee`, `liquidity`, sqrt-price endpoints, tick index, and a `steps` counter. The accessor functions used by integrators to read these fields after a swap are individually proven to return the field exactly.

Specifications: Swap Result Accessors

`pool::get_swap_result_a2b`

`pool::get_swap_result_amountified_remaining`

`pool::get_swap_result_amount_calculated`

`pool::get_swap_result_amountified`

`pool::get_swap_result_fee_amount`

`pool::get_swap_result_protocol_fee`

`pool::get_swap_result_liquidity`

`pool::get_swap_result_current_tick_index`

`pool::get_swap_result_end_sqrt_price`

`pool::get_swap_result_start_sqrt_price`

`pool::get_swap_result_starting_liquidity`

`pool::get_swap_result_steps`

`pool::get_swap_result_fee_growth_global`

`pool::get_swap_result_is_exceed`

`pool::get_swap_result_by_amount_in`

Concentrated Liquidity Math

The `clmm_math` module is the economic core of the protocol. It converts between token amounts, liquidity, and sqrt-prices, and it computes the input and output of every swap step. The specs here establish the solvency and rounding properties that the higher-level pool operations rely on.

Per-step swap solvency. The central never-pay-out-more-than-taken-in invariant is proven for `compute_swap_step`. A single step takes the input amount rounded up and pays the output amount rounded down on the opposite token over the same price range $[p_0, p_1]$. Valuing the output at the pool price and comparing against the input, the proof establishes:

$$\begin{aligned} \texttt{amount_out} \cdot 2^{128} &\le \texttt{amount_in} \cdot (p_0 \cdot p_1) && (\text{if } a2b) \\ \texttt{amount_out} \cdot (p_0 \cdot p_1) &\le \texttt{amount_in} \cdot 2^{128} && (\text{if } b2a) \end{aligned}$$

Both prices are valid sqrt-prices, so $p_0, p_1 \ge \texttt{min_sqrt_price} > 0$ and the price product $p_0 \cdot p_1$ is always strictly positive. For any single step the pool never pays out more value than it takes in.

Rounding direction. The liquidity-conversion helpers are proven to round in the solvent direction. `compute_liquidity_from_a` and `compute_liquidity_from_b` round down when crediting liquidity (never claiming more liquidity than the deposited amount funds) and round up when covering an amount (always covering the full deposit). The companion monotonicity specs prove the round-up result is always at least the round-down result, so a `div` / `div_round_up` swap in the body is caught.

Round-trip solvency. `get_liquidity_by_amount` converts a fixed token deposit into a liquidity value L by floor division. Feeding L back through `get_amount_by_liquidity` is proven to yield token amounts no greater than the original deposit:

$$\texttt{get_amount_by_liquidity}(\texttt{get_liquidity_by_amount}(\texttt{amount})$$

A user is never credited more liquidity than their deposit backs. This is the all-inputs generalisation of the module's single-point unit test.

Price movement and symmetry. The next-sqrt-price helpers are proven to move the price in the correct direction for a given input or output amount and to be monotone in the amount, so a larger trade moves the price further along the swap direction. The delta helpers are proven

symmetric under swapping the two price endpoints. The fee-rate denominator is proven to be exactly $1{,}000{,}000$.

Specifications: Concentrated Liquidity Math

`clmm_math::compute_swap_step`
`clmm_math::compute_swap_step_solvency`
`clmm_math::get_liquidity_by_amount_round_trip`
`clmm_math::compute_liquidity_from_a_rounding_solvency`
`clmm_math::compute_liquidity_from_b_rounding_solvency`
`clmm_math::compute_liquidity_from_a_rounding_monotonicity`
`clmm_math::compute_liquidity_from_b_rounding_monotonicity`
`clmm_math::compute_delta_a_rounding_monotonicity`
`clmm_math::get_next_sqrt_price_from_input`
`clmm_math::get_next_sqrt_price_from_output`
`clmm_math::compute_next_sqrt_price_b_down_monotonicity`
`clmm_math::get_delta_a_symmetry`
`clmm_math::get_delta_b_symmetry`
`clmm_math::fee_rate_denominator`

Tick Math and Signed Integers

The tick-to-price map and the signed-integer helper libraries sit beneath the pool, position, and tick logic. They were previously assumed correct, and are now verified.

Tick math converts between discrete tick indices and continuous **`sqrt_price`** values. These are the core price-mapping primitives that every swap, position, and liquidity operation depends on.

$$\texttt{get_sqrt_price_at_tick}(\texttt{tick}) = \sqrt{1.0001^{\texttt{tick}}}$$

The tick math functions involve conditional bit operations, magic constants, and iterative binary search that exceed the automated prover's SMT backend. The Sui Prover also supports a Lean 4 backend, which generates Lean models of Move code and specifications for interactive proof. We use this backend for tick math.

Lean 4 proofs (Sui Prover Lean backend)
The Lean backend is used wherever the automated SMT backend cannot decide a goal, and it carries a substantial share of this package: specifications across twelve of the nineteen specification modules are Lean-owned, including parts of the pool, gateway, administrative, and

timelock surfaces. The proofs described in this section are the tick-math ones specifically.

Each Move function has a corresponding Lean specification that mirrors the implementation structurally, using the same sequence of bit shifts, constants, and conditional multiplications. Types are modeled using bounded natural numbers that enforce bit-width constraints. Arithmetic primitives match those used in the Move implementation.

Floor tick correctness (`tick_at_sqrt_correctness`). For any valid `sqrt_price`, `get_tick_at_sqrt_price` returns the floor tick:

$$\texttt{get_sqrt_price_at_tick}(\texttt{tick}) \leq \texttt{sqrt_price} < \texttt{get_sqrt_price_at_tick}(\texttt{ti}$$

This guarantees that for every valid input, the returned tick places the price in the correct interval. A violation would let an attacker place liquidity at a different price than intended.

Monotonicity (`tick_at_sqrt_monotonic`). Higher sqrt-prices map to equal or higher ticks. Monotonicity failure would break the ordered tick indexing the protocol relies on to locate the active tick.

Round-trip inversion (`boundary_ticks_exact`). Converting a tick to its sqrt-price and back recovers the original tick exactly. Additionally, for any tick above `TICK_MIN`, the sqrt-price one unit below the boundary maps back to the previous tick. Together these establish that `get_sqrt_price_at_tick` partitions the valid sqrt-price range into contiguous, non-overlapping intervals indexed by tick, with `get_tick_at_sqrt_price` selecting the correct interval.

Bounds preservation. Valid inputs produce valid outputs in both directions, and the boundary values (`TICK_MIN`/`TICK_MAX`, `SQRT_PRICE_MIN`/`SQRT_PRICE_MAX`) map correctly.

Proof architecture

The proof proceeds in three layers.

Layer 1: Lean specifications. Each Move function is captured as a Lean definition with the same constants and control flow. For example, `get_sqrt_price_at_negative_tick` decomposes the tick into bit positions and conditionally multiplies by precomputed constants. The Lean definition mirrors this exactly:

Layer 2: Exhaustive verification. Two foundational properties (round-trip inversion and strict monotonicity of `get_sqrt_price_at_tick`) are verified by running the Lean spec functions at compile time over all 887,273 valid ticks (-443636 to 443636). Lean's `native_decide` tactic compiles and executes a tail-recursive boolean loop. The results are theorems consumed by the structural proofs rather than external test results, with the qualification that they rest on the correctness of Lean's compiler and evaluator in addition to its proof kernel:

Layer 3: Structural proofs. The main theorems are derived from the exhaustive results. The most involved is monotonicity of `get_tick_at_sqrt_price`. The algorithm computes two candidate ticks (`tick_low`, `tick_high`) differing by at most 1, then selects based on a sqrt-price comparison. The proof uses a case split showing that if the algorithm returns `tick_high` for a lower price but `tick_low` for a higher price, a contradiction arises:

Assumptions

The Lean proofs assume that intermediate arithmetic stays within bit-width bounds, consistent with the Move implementation's use of `u256` intermediate types.

One internal subroutine (the iterative log2 bit-refinement used to compute candidate ticks) is assumed monotonic rather than proven so via a loop invariant. This assumption is used only in the structural monotonicity proof. The exhaustive `native_decide` verification independently confirms the end-to-end correctness and monotonicity of both functions for every concrete valid input. The axiom bridges the gap between "verified for all values" and "proven structurally" rather than introducing unvalidated trust.

We also prove `is_valid_index` validates tick alignment (range and spacing divisibility) and boundary constants (`min_tick`, `max_tick`, `min_sqrt_price`, `max_sqrt_price`, `tick_bound`).

Signed integer helpers. The `i128H`, `i64H`, and `i32H` libraries wrap the protocol's signed-integer representation. Each arithmetic and comparison operation is proven to agree with the reference signed-integer semantics, with overflow asserts pinning each result to the signed range. For example, addition is proven to satisfy:

$$\mathbf{signed}(\mathbf{add}(a, b)) = \mathbf{signed}(a) + \mathbf{signed}(b)$$

under the assertion that the sum fits the signed width. The conversion functions between the protocol's representation and the math-library representation are proven to round-trip.

Specifications: Tick Math and Signed Integers

`tick_math::get_sqrt_price_at_tick`

`tick_math::is_valid_index`

`tick_math::min_tick`

`tick_math::max_tick`

`tick_math::min_sqrt_price`

`tick_math::max_sqrt_price`

`tick_math::tick_bound`

`i128H::add`

`i128H::sub`

`i128H::neg`

`i128H::lib_to_mate`

`i128H::mate_to_lib`

`i64H::add`

`i64H::sub`

`i32H::add`

`i32H::sub`

Arithmetic Operations

The protocol's liquidity arithmetic centres on `utils::add_delta`, which applies a signed `I128` delta to an unsigned `u128` liquidity value. The spec asserts non-underflow on subtraction and non-overflow on addition, computes the exact signed sum, and proves the result is strictly below `u128::MAX`:

$$\texttt{add_delta}(L,\,\delta) = \begin{cases} L - |\delta| & \text{if } \delta < 0 \text{ and } L \geq |\delta| \\ L + |\delta| & \text{if } \delta \geq 0 \text{ and } |\delta| < 2^{128} - L \end{cases}$$

This primitive is the single point at which the prover discharges the arithmetic side-condition for every liquidity mutation, both at the pool level and inside each tick boundary.

The constants used throughout the protocol are each proven to evaluate to their documented values.

Specifications: Arithmetic and Constants

`utils::add_delta`

`utils::deposit_balance`
`utils::withdraw_balance`
`utils::transfer_balance`
`utils::transfer_coin`
`utils::timestamp_seconds`
`utils::get_type_string`
`utils::u128_to_string`
`constants::max_u8`
`constants::max_u16`
`constants::max_u32`
`constants::max_u64`
`constants::max_u128`
`constants::max_u256`
`constants::q64`
`constants::max_allowed_fee_rate`
`constants::max_allowed_tick_spacing`
`constants::max_protocol_fee_share`
`constants::max_observation_cardinality`
`constants::protocol_fee_share`
`constants::manager`
`constants::blue_reward_type`
`constants::pool_creation_fee_dynamic_key`

Conclusion and Continual Assurance

This verification effort covers the bulk of Bluefin CLMM's on-chain surface. Pool lifecycle, the concentrated-liquidity math layer, position and reward accounting, the tick state primitives, the tick-to-price map, the TWAP observation buffer, and the capability-gated administrative surface are all verified.

The proofs establish that a single swap step never pays out more value than it takes in, and that liquidity conversion rounds in the solvent direction and round-trips without overcrediting. Pool state evolves coherently, ticks maintain the signed net liquidity invariant, positions track fees and rewards exactly, the oracle preserves its buffer invariants, and governance applies its intended effects under valid capability and version checks.

A minority of specifications establish their properties of executions that complete rather than also characterising exactly when the call reverts. Some are mathematical lemmas for which that is the natural form. The others are the composed swap and liquidity-provision paths, where an exact revert characterisation was attempted, shown false by machine-checked counterexample, and deliberately not claimed. Reducing that set is active work. Their building blocks are independently verified, so the solvency, rounding, and state-coherence guarantees those entry points compose are themselves proven. All such cases are tracked in the verification dashboard in greater detail.

These results apply to the current deployed contract revision and specification set captured in this report. Any protocol upgrade, parameterization change that affects verified logic, or dependency-level semantic change should be treated as a trigger for re-running and refreshing the formal proofs, so verification remains a continuous assurance process rather than a one-time milestone.

This report reflects the state of the specifications and proofs as of the revision and prover version listed in the Reproducibility section. Formal verification reduces the risk of certain classes of bugs but does not guarantee the absence of all vulnerabilities; only explicitly specified properties are verified. This report is provided for informational purposes only, not as financial, legal, or investment advice. No liability is assumed for losses arising from reliance on this report. Users should perform their own due diligence before interacting with any smart contract system.

Appendix: Lean 4 Models

Tick-to-price: get_sqrt_price_at_negative_tick

Conditional fixed-point multiplications by precomputed constants for each of 18 bit positions. Each constant encodes 1.0001^{-2^k} scaled to Q64.64 fixed-point. Two of 18 bit positions shown.

MOVE

```
fun get_sqrt_price_at_negat
    let abs_tick = i32::as_
    let mut ratio = if (abs
        18446582180567539231
    } else {
        18446744073709551616
    };
    if (abs_tick & 0x2 != 0
        ratio = full_math_u
    };
    // ... 16 more bit posi
```

LEAN 4

```
def get_sqrt_price_at_negat
  let abs_tick := Int_ops.c
  let ratio := if (Int_ops.
                 != (Int_c
    (18446582180567539231 :
  else
    (18446744073709551616 :
  let ratio := if (Int_ops.
                 != (Int_c
    Full_math_u128.mul_shr
      (18446899583751176498
      (64 : BoundedNat (2^8
  else ratio
  -- ... 16 more bit positi
```

Exhaustive monotonicity: native_decide over 887,272 consecutive tick pairs

The first theorem checks all consecutive pairs at compile time. The second lifts this to arbitrary valid tick pairs by induction on the gap between ticks, reducing to the verified consecutive-pair case.

MOVE

```
// The log2 refinement loop
// squaring and bit extract
// fixed-point precision
let mut shift = 31;
while (shift >= 18) {
    r = ((r * r) >> 63);
    f = ((r >> 64) as u8);
    log_2_x32 = i128::or(lo
        i128::shl(i128::fro
    r = r >> f;
    shift = shift - 1;
};
```

LEAN 4

```
-- Strict monotonicity: sqr
private theorem check_conse
    checkAll (fun n =>
      ((get_sqrt_price_at_t
       (get_sqrt_price_at_t
      887272 = true := by r

-- Lift to arbitrary valid
theorem sqrt_at_tick_strict
    (h1 : valid_tick t1) (h
    get_sqrt_price_at_tick_
  strict_mono_of_consecutiv
```

Monotonicity of get_tick_at_sqrt_price: case-split contradiction

The algorithm returns either tick_low or tick_high (gap at most 1). If it chose tick_high for a lower price but tick_low for a higher price, the selection conditions yield a contradiction.

MOVE

```
// The tick selection logic
// two candidate ticks diff
let tick_low = i128::as_i32
    i128::sub(log_sqrt_1000
let tick_high = i128::as_i3
    i128::add(log_sqrt_1000

if (i32::eq(tick_low, tick_
    tick_low
} else if (get_sqrt_price_c
    tick_high
} else {
    tick_low
}
```

LEAN 4

```
theorem tick_at_sqrt_monotc
    (h1 : valid_sqrt_price
    (h_le : sqrt_price1 ≤ s
    get_tick_at_sqrt_price_
  -- ...establish monotonic

  -- Critical case: algorit
  -- sqrt_price2. Then sqrt
  -- sqrt_price2 < sqrt_at_
  -- sqrt_price2 < sqrt_at_
  omega
```